Arquitectura matemática del código

Aplicación del control óptimo de campo medio a clasificación binaria
Basado en Daudin & Delarue, *arXiv:2507.08486*

5 de abril de 2026

Índice

1. Datos: generación y preprocesado **2**
1.1. Generación del conjunto `make_moons` . . . 2
1.2. Normalización (`StandardScaler`) . . . 2

2. Arquitectura del modelo **2**
2.1. Campo vectorial prototípico . . . 2
2.2. Campo efectivo de campo medio . . . 3
2.3. ODE de transporte y flujo . . . 3
2.4. Integración numérica: Runge-Kutta de orden 4 . . . 4
2.5. Clasificador lineal . . . 4

3. Función de pérdida **4**
3.1. Coste total . . . 4
3.2. Entropía cruzada binaria (BCE) . . . 5
3.3. Penalización entrópica . . . 5

4. Entrenamiento **6**
4.1. Optimizador Adam . . . 6
4.2. SGD con tasa de aprendizaje constante . . . 6
4.3. Stochastic Gradient Langevin Dynamics (SGLD) . . . 6
4.4. Cosine Annealing . . . 7
4.5. Gradient Clipping . . . 7
4.6. Retropropagación . . . 7
4.7. Hiperparámetros de entrenamiento . . . 7

5. Fundamentos teóricos **7**
5.1. Condición Polyak-Łojasiewicz (PL) . . . 7
5.2. Genericidad del minimizador . . . 8

6. Descripción matemática de los experimentos **8**
6.1. Experimento A: Evolución de γ_t . . . 8
6.2. Experimento B: Efecto del parámetro ε . . . 9
6.3. Experimento C: Verificación de la condición PL . . . 9
6.4. Experimento D: Genericidad del minimizador . . . 10
6.5. Experimento E: Robustez de ν^* (make_moons) . . . 10
6.6. Experimento F: Simetría de ν^* en `make_circles` . . . 10

7. Resumen de la arquitectura **11**

1. Datos: generación y preprocesado

1.1. Generación del conjunto `make_moons`

Se generan $N = 400$ pares (x_i, y_i) con la función `make_moons` de `scikit-learn`. Cada punto pertenece a una de dos clases entrelazadas: $y_i \in \{0, 1\}$, con 200 puntos por clase.

A las coordenadas limpias se les añade ruido gaussiano independiente en cada dimensión:

$$x_i \leftarrow x_i + \eta_i, \qquad \eta_i \sim \mathcal{N}(0, \sigma^2 I_2), \quad \sigma = 0{,}12. \tag{1}$$

1.2. Normalización (`StandardScaler`)

Tras añadir el ruido, se aplica normalización estándar componente a componente. Sea μ_k y s_k la media y desviación típica de la componente k sobre los N puntos. La transformación es:

$$\tilde{x}_i^{(k)} = \frac{x_i^{(k)} - \mu_k}{s_k}, \qquad k = 1, 2. \tag{2}$$

Tras la normalización, cada coordenada tiene media 0 y varianza 1. Esto estabiliza el entrenamiento al colocar los datos en una escala homogénea.

En adelante se denota $x_0^{(i)} \in \mathbb{R}^2$ al punto normalizado i-ésimo y $\gamma_0 = \frac{1}{N}\sum_{i=1}^{N} \delta_{(x_0^{(i)}, y_i)}$ a la medida empírica de los datos de entrenamiento.

2. Arquitectura del modelo

El modelo es una *Mean-Field ResNet* compuesta de dos bloques:

1. Un **campo vectorial de campo medio** (`MeanFieldVelocity`) que define la dinámica de una ODE,
2. un **clasificador lineal** que opera sobre el estado final de la ODE.

2.1. Campo vectorial prototípico

Siguiendo el Ejemplo 1.1 del paper [1], se utiliza el campo vectorial prototípico

$$b(x, a) = \sigma(a_1 \cdot x + a_2)\, a_0, \qquad \sigma = \tanh, \tag{3}$$

donde el *parámetro de control* de cada neurona m es

$$a^m = (a_0^m,\, a_1^m,\, a_2^m,\, b_1^m) \;\in\; \mathbb{R}^2 \times \mathbb{R}^2 \times \mathbb{R} \times \mathbb{R} \;=\; \mathbb{R}^6. \tag{4}$$

Los roles de cada componente son:

- $a_1^m \in \mathbb{R}^2$: *pesos de entrada*; definen la dirección del espacio de features que la neurona m "proyecta". La pre-activación es $a_1^m \cdot x \in \mathbb{R}$.
- $a_2^m \in \mathbb{R}$ (columna temporal de W_1): coeficiente que escala la contribución explícita del tiempo $t \in [0, T]$ a la pre-activación.
- $b_1^m \in \mathbb{R}$: sesgo fijo de la neurona m en $t = 0$.
- $a_0^m \in \mathbb{R}^2$: *pesos de salida*; escalan la contribución de la neurona al campo vectorial en $\mathbb{R}^2$. Su norma $\|a_0^m\|_2$ mide la *importancia* de la neurona.

2.2. Campo efectivo de campo medio

En la aproximación de M partículas (neuronas), la medida de control $\nu_t \approx \frac{1}{M}\sum_{m=1}^{M}\delta_{a^m}$ produce el campo efectivo:

$$F(x,t) = \frac{1}{M}\sum_{m=1}^{M} b(x, a^m(t)) = \frac{1}{M}\sum_{m=1}^{M} \sigma\big(a_1^m \cdot x + a_2^m\, t + b_1^m\big)\, a_0^m. \tag{5}$$

Diferencia con el paper. El paper optimiza sobre trayectorias arbitrarias $(a^m(t))_{t\in[0,T]}$. En la implementación, los parámetros (a_1^m, a_0^m, b_1^m) son **estáticos** (independientes de t), y la dependencia temporal se introduce *explícitamente* añadiendo t como entrada adicional:

$$\text{pre-activación}^m(x,t) = a_1^m \cdot x + a_2^m \cdot t + b_1^m. \tag{6}$$

Esto restringe la familia de controles a aquellos cuya trayectoria $a_2^m \mapsto a_2^m \cdot t + b_1^m$ varía *linealmente* en t, pero preserva la dependencia temporal del campo $F(x,t)$.

Implementación matricial. Con $M = 64$ neuronas y $d_1 = 2$:

$$W_1 \in \mathbb{R}^{M\times(d_1+1)}, \quad [W_1]_{m,:} = (a_1^m,\, a_2^m), \quad b_1 \in \mathbb{R}^M, \tag{7}$$

$$W_0 \in \mathbb{R}^{d_1\times M}, \quad [W_0]_{:,m} = a_0^m \quad \text{(sin bias)}. \tag{8}$$

Para N puntos simultáneos $X \in \mathbb{R}^{N\times d_1}$:

$$h = \tanh\big([\,X,\ t\,\mathbf{1}\,]\,W_1^\top + b_1\big) \ \in\ \mathbb{R}^{N\times M}, \qquad F(X,t) = \frac{1}{M}\, h\, W_0^\top \ \in\ \mathbb{R}^{N\times d_1}. \tag{9}$$

El factor $1/M$ se absorbe en la escala de inicialización de W_0.

Total de parámetros de la ODE.

$$\underbrace{M\cdot(d_1+1)}_{\text{pesos } W_1} + \underbrace{M}_{\text{bias } b_1} + \underbrace{d_1\cdot M}_{\text{pesos } W_0} = 64\cdot 3 + 64 + 2\cdot 64 = 384. \tag{10}$$

2.3. ODE de transporte y flujo

El estado $X_t \in \mathbb{R}^2$ de cada punto de dato evoluciona según la ODE de primer orden:

$$\frac{dX_t}{dt} = F(X_t, t), \qquad X_0 = x_0, \qquad t\in[0,T], \quad T = 1. \tag{11}$$

La solución define el *flujo* $\phi_t : \mathbb{R}^2 \to \mathbb{R}^2$ con $X_t = \phi_t(x_0)$. La distribución de features evoluciona como la *medida push-forward*:

$$\gamma_t = (\phi_t)_{\#}\gamma_0, \tag{12}$$

lo que es equivalente a la ecuación de continuidad (ec. 1.3 del paper):

$$\partial_t \gamma_t + \operatorname{div}_x\big(F(x,t)\,\gamma_t\big) = 0. \tag{13}$$

Esta PDE garantiza conservación de masa: los puntos no se crean ni se destruyen, sino que son *transportados* por el campo F.

2.4. Integración numérica: Runge-Kutta de orden 4

La ODE (11) se integra con $n_{\text{steps}} = 10$ pasos de Runge-Kutta 4 (RK4), con paso $\Delta t = T/n_{\text{steps}} = 0{,}1$. En cada paso k (con $t_k = k \cdot \Delta t$):

$$k_1 = F(X_{t_k},\ t_k), \tag{14}$$

$$k_2 = F\big(X_{t_k} + \tfrac{\Delta t}{2}\, k_1,\ t_k + \tfrac{\Delta t}{2}\big)\,, \tag{15}$$

$$k_3 = F\big(X_{t_k} + \tfrac{\Delta t}{2}\, k_2,\ t_k + \tfrac{\Delta t}{2}\big)\,, \tag{16}$$

$$k_4 = F(X_{t_k} + \Delta t\, k_3,\ t_k + \Delta t)\,, \tag{17}$$

$$X_{t_{k+1}} = X_{t_k} + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4). \tag{18}$$

El error global de RK4 es $\mathcal{O}(\Delta t^4) = \mathcal{O}(10^{-4})$, cuatro órdenes de magnitud mejor que Euler ($\mathcal{O}(\Delta t) = 0{,}1$), al coste de cuatro evaluaciones de F por paso.

2.5. Clasificador lineal

Tras integrar la ODE hasta $t = T$, se obtiene el estado final $X_T \in \mathbb{R}^{N\times 2}$. El clasificador lineal calcula el logit:

$$\ell_i = W \cdot X_T^{(i)} + b, \qquad W \in \mathbb{R}^{1\times 2},\ b \in \mathbb{R}, \tag{19}$$

y la probabilidad predicha es:

$$\hat{p}_i = \mathbb{P}(y_i = 1 \mid X_T^{(i)}) = \sigma(\ell_i) = \frac{1}{1 + e^{-\ell_i}}. \tag{20}$$

Parámetros del clasificador.

$$\underbrace{1 \times 2}_{\text{pesos } W} + \underbrace{1}_{\text{bias } b} = 3 \text{ parámetros}, \tag{21}$$

lo que representa menos del 1 % del total de parámetros del modelo (387).

Intuición. La ODE aprende a *transportar* la distribución γ_0 (lunas entrelazadas, no separables linealmente) hasta γ_T (clases separadas). El clasificador lineal simplemente lee el resultado: toda la complejidad no lineal está en el flujo.

3. Función de pérdida

3.1. Coste total

La función de pérdida que se minimiza es:

$$J(\theta) = \underbrace{\mathcal{L}_{\text{BCE}}(\theta)}_{\text{coste terminal}} + \varepsilon \cdot \underbrace{\mathcal{E}(\theta)}_{\text{penalización entrópica}}, \tag{22}$$

donde θ denota el conjunto completo de parámetros (W_1, b_1, W_0, W, b) y $\varepsilon > 0$ es el coeficiente de regularización.

3.2. Entropía cruzada binaria (BCE)

El coste terminal es la entropía cruzada binaria (Binary Cross-Entropy) con logits:

$$\mathcal{L}_{\mathrm{BCE}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log\sigma(\ell_i) + (1-y_i)\log(1-\sigma(\ell_i))\Big], \tag{23}$$

donde $\ell_i = W \cdot X_T^{(i)} + b$ y $\sigma(\ell_i) = (1+e^{-\ell_i})^{-1}$. En términos del paper [1], este es el coste terminal $L(X_T, Y)$ del problema de control óptimo:

$$\min_{\nu_t} \int_0^T \varepsilon \cdot E(\nu_t \mid \nu^\infty)\, dt + \mathbb{E}\big[L(X_T, Y)\big], \tag{24}$$

donde $E(\nu_t \mid \nu^\infty)$ es la divergencia KL relativa al prior ν^∞.

3.3. Penalización entrópica

Motivación teórica. El paper regula la medida de control ν_t mediante la divergencia KL respecto al prior de Gibbs:

$$\nu^\infty \propto \exp(-\ell(a)), \qquad \ell(a) = c_1\,|a|^4 + c_2\,|a|^2\,, \quad c_1 = 0{,}05,\ c_2 = 0{,}5. \tag{25}$$

El término cuártico $c_1\,|a|^4$ es esencial: satisface la condición de *supercoercividad* ($\nabla^2\ell(a) \geq c(1+|a|^2)I$) que garantiza la desigualdad log-Sobolev para ν^∞ y por tanto la condición Polyak-Łojasiewicz (véase Sección 5.1). Un potencial puramente cuadrático ($c_1 = 0$) no sería suficiente.

La KL completa es:

$$\mathrm{KL}(\nu_t \| \nu^\infty) = \underbrace{\mathbb{E}_{\nu_t}[\ell(a)]}_{\text{término de energía}} - \underbrace{H(\nu_t)}_{\text{entropía diferencial}}. \tag{26}$$

Limitación de la implementación. En la implementación, $\nu_t = \frac{1}{M}\sum_{m=1}^{M}\delta_{\theta^m}$ es una suma de deltas de Dirac sobre los parámetros puntuales. La entropía diferencial de una medida discreta respecto a un prior continuo es $-\infty$, por lo que la KL completa no es computable.

Sólo es accesible el **término de energía**:

$$\mathcal{E}(\theta) = \frac{1}{N_p}\sum_{j=1}^{N_p}\Big[c_1\,\theta_j^4 + c_2\,\theta_j^2\Big], \quad c_1 = 0{,}05,\ c_2 = 0{,}5, \tag{27}$$

donde la suma recorre los $N_p = 384$ parámetros de `MeanFieldVelocity` (se *excluyen* los 3 parámetros del clasificador, que son parte del coste terminal, no del control). Dividir por N_p hace que el valor no dependa del tamaño del modelo.

Observación 3.1. En términos prácticos, $\mathcal{E}(\theta)$ es equivalente a regularización $L^4 + L^2$ (*weight decay* polinomial). Este término proporciona el **prior energético** ν^∞, pero no el término de entropía $-H(\nu_t)$ de la KL completa. Para los experimentos B, E y F, el término de entropía se implementa mediante **SGLD** (ruido de Langevin en el gradiente, véase Sección 4.3).

Estructura de a^m para la penalización. Recordando que cada neurona m tiene parámetro $a^m = (a_0^m, a_1^m, a_2^m, b_1^m) \in \mathbb{R}^6$, la norma que aparece en $\ell(a)$ es:

$$|a^m|^2 = \|a_0^m\|^2 + \|a_1^m\|^2 + (a_2^m)^2 + (b_1^m)^2. \tag{28}$$

En la implementación, $\mathcal{E}$ se calcula recorriendo todos los parámetros escalares θ_j de `MeanFieldVelocity` de forma unificada.

4. Entrenamiento

La función `train()` implementa tres modos de optimización, cada uno adecuado a un experimento distinto:

Modo	Parámetro	Experimentos	Objetivo
Adam + cosine annealing	(defecto)	A, D	Convergencia rápida
SGD + lr constante	`use_sgd=True`	C	Verificación PL sin artefactos
SGLD + cosine annealing	`use_sgld=True`	B, E, F	Muestreo de ν_t^*

4.1. Optimizador Adam

Se utiliza el optimizador Adam [3] con tasa de aprendizaje $\alpha_0 = 0{,}01$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon_{\text{Adam}} = 10^{-8}$. En cada paso de gradiente s, para cada parámetro escalar θ:

$$g_s = \nabla_\theta J(\theta^{s-1}), \tag{29}$$

$$m_s = \beta_1 m_{s-1} + (1-\beta_1)\, g_s \qquad \text{(primer momento, media exponencial)}, \tag{30}$$

$$v_s = \beta_2 v_{s-1} + (1-\beta_2)\, g_s^2 \qquad \text{(segundo momento, varianza exponencial)}, \tag{31}$$

$$\hat{m}_s = \frac{m_s}{1-\beta_1^s} \qquad \text{(corrección de sesgo)}, \tag{32}$$

$$\hat{v}_s = \frac{v_s}{1-\beta_2^s}, \tag{33}$$

$$\theta^s = \theta^{s-1} - \alpha_s \frac{\hat{m}_s}{\sqrt{\hat{v}_s} + \varepsilon_{\text{Adam}}}. \tag{34}$$

La corrección de sesgo es importante en los primeros pasos, cuando $m_s \approx (1-\beta_1)g_1$ y $v_s \approx (1-\beta_2)g_1^2$ son aún pequeños. Sin corrección, el paso efectivo sería demasiado grande.

4.2. SGD con tasa de aprendizaje constante

Para el Experimento C (verificación de la condición PL), se usa SGD puro con tasa de aprendizaje *constante* $\alpha = 0{,}01$ y sin ruido:

$$\theta^s = \theta^{s-1} - \alpha\, \nabla_\theta J(\theta^{s-1}). \tag{35}$$

La motivación es que la condición PL es una propiedad **geométrica** del paisaje de J, independiente del optimizador. Con cosine annealing, $\alpha_s \to 0$ al final aplana $J(\theta^s) - J^*$ artificialmente, lo que contamina la estimación de $\hat{\mu}$. Con lr constante, J^* es el mínimo geométrico real del gradient flow.

4.3. Stochastic Gradient Langevin Dynamics (SGLD)

Para los Experimentos B, E y F, se usa SGLD [2]: la discretización de Euler-Maruyama de la SDE

$$d\theta_t = -\nabla J(\theta_t)\, dt + \sqrt{2\varepsilon}\, dW_t, \tag{36}$$

donde W_t es un movimiento Browniano estándar. El esquema discreto con paso η_s (tasa de aprendizaje en el paso s) es:

$$\theta^s = \theta^{s-1} - \eta_s\, \nabla J(\theta^{s-1}) + \sqrt{2\,\eta_s\,\varepsilon}\, \xi^s, \qquad \xi^s \sim \mathcal{N}(0, I). \tag{37}$$

El ruido $\sqrt{2\eta_s\varepsilon}\,\xi^s$ implementa el **término de entropía** $-H(\nu_t)$ de la KL: hace que los parámetros exploren la distribución de Gibbs $\nu^* \propto \exp(-J(\theta)/\varepsilon)$ en lugar de colapsar a un estimador puntual (MAP). El término de energía $\mathcal{E}(\theta)$ en (27) actúa como el prior ν^∞.

Con $\varepsilon = 0$ el ruido es exactamente cero, recuperando SGD estándar. La escala $\sqrt{2\eta_s\varepsilon}$ está derivada de la teoría del paper (sección 1.3): ε es la temperatura entrópica y η_s el paso de Euler-Maruyama. El ruido se inyecta *después* del paso de gradiente y *antes* del scheduler (para usar el lr actual η_s, no el siguiente).

La tasa de aprendizaje sigue cosine annealing ($\eta_s \to 0$), lo que implementa *simulated annealing* natural y satisface las condiciones de Robbins-Monro: $\sum_s \eta_s = \infty$, $\sum_s \eta_s^2 < \infty$.

4.4. Cosine Annealing

La tasa de aprendizaje α_s no es constante: sigue un calendario de *cosine annealing* [4] con $\eta_{\text{mín}} = 0$ y $T_{\text{máx}} = n_{\text{epochs}}$:

$$\alpha_s = \frac{\alpha_0}{2}\Big(1 + \cos\Big(\frac{\pi s}{T_{\text{máx}}}\Big)\Big), \qquad s = 0, 1, \ldots, T_{\text{máx}}. \tag{38}$$

La tasa arranca en $\alpha_0 = 0{,}01$, decrece suavemente hasta 0 al final del entrenamiento. En Adam (experimentos A, D) y SGLD (B, E, F), el aplanamiento de las curvas de pérdida en las últimas épocas se debe al scheduler, no a una violación de la condición PL. Por eso el Experimento C usa SGD con lr constante (Sección 4.2).

4.5. Gradient Clipping

Para evitar gradientes explosivos en las primeras épocas (cuando los parámetros son aún aleatorios y el campo F puede ser grande), se aplica *gradient clipping* por norma global:

$$g \leftarrow \begin{cases} g & \text{si } \|g\| \le G_{\text{máx}}, \\ G_{\text{máx}} \dfrac{g}{\|g\|} & \text{si } \|g\| > G_{\text{máx}}, \end{cases} \qquad G_{\text{máx}} = 5. \tag{39}$$

Aquí $g = \nabla_\theta J(\theta)$ es el vector de gradientes concatenado sobre todos los parámetros, y la norma es la euclídea global. El clipping escala el vector entero (no componente a componente), preservando la dirección del gradiente.

En Adam, el segundo momento $\hat{v}_s$ hace de divisor adaptativo, pero sin clipping la actualización puede ser muy grande en la primera época cuando $\hat{v}_s$ todavía no ha acumulado información.

4.6. Retropropagación

El gradiente $\nabla_\theta J$ se calcula automáticamente con `loss.backward()` de PyTorch. El grafo computacional incluye:

1. Diferenciación a través del clasificador lineal ($\partial J/\partial W$, $\partial J/\partial b$).
2. Diferenciación a través del RK4 (cuatro evaluaciones de F por paso, $n_{\text{steps}} = 10$ pasos): $\partial J/\partial X_T \cdot \partial X_T/\partial\theta$ mediante la regla de la cadena.
3. Diferenciación de la penalización entrópica $\partial\mathcal{E}/\partial\theta_j = 4c_1\theta_j^3 + 2c_2\theta_j$.

PyTorch construye el grafo dinámicamente en el paso *forward*, por lo que no es necesario implementar el adjunto del integrador manualmente.

4.7. Hiperparámetros de entrenamiento

5. Fundamentos teóricos

5.1. Condición Polyak-Łojasiewicz (PL)

El paper [1] demuestra (Meta-Teorema 2) que para $\varepsilon > 0$ y condiciones iniciales en un conjunto abierto y denso $\mathcal{O}$, la función de pérdida satisface la *desigualdad de Polyak-Łojasiewicz*

Hiperparámetro	Valor	Experimentos	Descripción
N	400	todos	Puntos de entrenamiento
d_1	2	todos	Dimensión del espacio de features
M	64	todos	Neuronas / partículas
T	1.0	todos	Horizonte temporal de la ODE
n_{steps}	10	todos	Pasos RK4
ε	0.01	todos	Temperatura entrópica / regularización
α_0	0.01	todos	Tasa de aprendizaje inicial
β_1, β_2	0.9, 0.999	A, D	Momentos de Adam
n_{epochs}	800 (A), 700 (B–F)	todos	Épocas de entrenamiento
$G_{\text{máx}}$	5.0	todos	Umbral de gradient clipping
c_1	0.05	todos	Coeficiente cuártico del prior
c_2	0.5	todos	Coeficiente cuadrático del prior
Solo SGLD (experimentos B, E, F):			
$\sqrt{2\eta_s\varepsilon}$	variable	B, E, F	Escala del ruido de Langevin

Cuadro 1: Hiperparámetros del modelo y el entrenamiento.

(PL) local:

$$\|\nabla J(\theta)\|^2 \;\geq\; 2\mu \cdot (J(\theta) - J^*), \qquad \mu > 0, \tag{40}$$

donde $J^* = \inf_\theta J(\theta)$ es el valor óptimo. Esta condición implica convergencia *exponencial* del gradient descent con tasa de aprendizaje η:

$$J(\theta^s) - J^* \;\leq\; (1 - 2\eta\mu)^s \cdot (J(\theta^0) - J^*). \tag{41}$$

Observación 5.1. La condición PL es más débil que la convexidad fuerte: no implica que el espacio de parámetros sea convexo ni que el mínimo sea único en sentido geométrico, pero sí garantiza que cualquier punto estacionario es global. La clave es que $\varepsilon > 0$: con $\varepsilon = 0$ no hay garantía.

5.2. Genericidad del minimizador

El Meta-Teorema 1 del paper establece que para un conjunto abierto y denso $\mathcal{O}$ de condiciones iniciales γ_0, el problema de control tiene un único minimizador *estable*. Esto predice robustez empírica: distintas inicializaciones de parámetros y distintos datasets deberían producir el mismo J^* (a igualdad de clase de problema).

6. Descripción matemática de los experimentos

6.1. Experimento A: Evolución de γ_t

Objetivo. Visualizar que la ODE transforma γ_0 (lunas no separables) en γ_T (clases separadas).

Protocolo. Se entrena un modelo con $\varepsilon = 0{,}01$, $M = 64$, $T = 1$, $n_{\text{epochs}} = 800$. Tras el entrenamiento, se integra la ODE guardando X_t en $t \in \{0, T/4, T/2, 3T/4, T\}$. Se visualiza la distribución γ_t en cada instante.

Magnitudes visualizadas.

- Snapshots γ_t para $t \in \{0, 0{,}25, 0{,}5, 0{,}75, 1{,}0\}$.
- Trayectorias $t \mapsto X_t^{(i)}$ para 40 partículas seleccionadas.

- Curvas de convergencia: J, BCE, $\mathcal{E}$ vs. época.
- Frontera de decisión final en $\mathbb{R}^2$.

Resultado esperado. γ_T debe ser linealmente separable con accuracy $\approx 1{,}0$.

6.2. Experimento B: Efecto del parámetro ε

Objetivo. Estudiar el efecto del coeficiente de regularización entrópica sobre la convergencia, las fronteras de decisión y el campo de velocidad.

Protocolo. Se entrena un modelo para cada $\varepsilon \in \{0, 10^{-3}, 10^{-2}, 0{,}1, 0{,}5\}$ con **SGLD** (`use_sgld=True`), manteniendo la misma semilla de inicialización. El uso de SGLD es coherente con el rol de ε como temperatura entrópica: el ruido Langevin $\sqrt{2\eta_s\varepsilon}\,\xi$ hace que los parámetros exploren $\nu^* \propto \exp(-J(\theta)/\varepsilon)$. Con $\varepsilon = 0$ el ruido es exactamente cero (SGD puro).

Magnitudes analizadas.

- **B1 — Curvas de convergencia:** J total, accuracy y $\mathcal{E}$ vs. época para cada ε. Predicción: J^* crece con ε (la regularización penaliza la solución óptima "pura"), pero todas las ε convergen a accuracy $\approx 1{,}0$.
- **B2 — Fronteras de decisión y campo de velocidad:** la fila superior muestra la región de decisión $\{x : \hat{p}(x) \geq 0{,}5\}$ para cada ε; la fila inferior muestra el quiver plot del campo $F(x, t = 0{,}5)$ normalizado en dirección (color = magnitud). Predicción: fronteras más suaves para ε mayor; campo más regular.

Trade-off empírico ε vs. $\hat{\mu}_{\mathbf{PL}}$. Empíricamente se observa que $\hat{\mu}_{\mathrm{PL}}$ no crece monotónicamente con ε. Esto no contradice el paper: la teoría garantiza $\mu > 0$ para todo $\varepsilon > 0$, pero no que μ sea monótono. Con ε grande, J^* es mayor (más penalización) y el exceso de coste $(J - J^*)$ en el denominador de la ratio PL puede crecer más que $\|\nabla J\|^2$, reduciendo $\hat{\mu}$.

6.3. Experimento C: Verificación de la condición PL

Objetivo. Verificar empíricamente la desigualdad (40).

Protocolo. Experimento **autocontenido**: entrena sus propios modelos con **SGD + lr constante** (`use_sgd=True`, $\varepsilon \in \{0, 10^{-3}, 10^{-2}, 0{,}1, 0{,}5\}$, 700 épocas). No reutiliza modelos del Experimento B. La motivación es que la condición PL es una propiedad **geométrica** del paisaje de J, independiente del optimizador: con cosine annealing, $\alpha_s \to 0$ aplana $\delta_s = J(\theta^s) - J^*$ artificialmente y contamina $\hat{\mu}$. Con lr constante, J^* es el mínimo geométrico genuino del gradient flow.

En cada época s, se registran:

$$\text{excess cost}: \quad \delta_s = J(\theta^s) - J^*, \qquad \text{grad norm}: \quad G_s = \|\nabla J(\theta^s)\|^2 . \tag{42}$$

El estimador conservador de μ se define como el percentil 10 del ratio

$$\hat{\mu}_{\mathrm{PL}} = \mathrm{P}_{10}\left\{ \frac{G_s}{2\,\delta_s} : s = 1, \ldots, n_{\text{epochs}} \right\}. \tag{43}$$

Paneles generados.

- **C1 — Log-log:** G_s vs. δ_s. Los puntos deben quedar por encima de la recta $G = 2\hat{\mu}\,\delta$.
- **C2 — Semilog:** δ_s vs. época. Caída exponencial confirma la convergencia predicha por (40).
- **C3 — Barras:** $\hat{\mu}_{\mathrm{PL}}$ para cada ε. Confirma $\hat{\mu} > 0$ para todo $\varepsilon > 0$.
- **C4 — Ratio PL:** $G_s/(2\delta_s)$ vs. época (escala log), que debe mantenerse $\geq \hat{\mu}$ en todo momento.

6.4. Experimento D: Genericidad del minimizador

Objetivo. Verificar empíricamente el Meta-Teorema 1: que el minimizador es robusto a cambios en la inicialización y en el dataset.

Protocolo. Se ejecutan tres sub-experimentos con $n_{\text{seeds}} = 10$, 500 épocas, Adam y $\varepsilon \in \{0, 0{,}01\}$:

- **D1 — γ_0 fija, inicialización variable:** `data_seed`$= 42$, `init_seed`$\in \{0, \ldots, 9\}$.
- **D2 — Inicialización fija, γ_0 variable:** `init_seed`$= 4$, `data_seed`$\in \{0, \ldots, 9\}$.
- **D3 — Ambas aleatorias:** `data_seed`=`init_seed`$= s$.

Resultado esperado. D3 tiene la banda de pérdida más ancha (combina ambas fuentes de variabilidad). Con $\varepsilon = 0{,}01$, la condición PL garantiza $\hat{\mu} > 0$ para todo run, pero la regularización no reduce necesariamente la varianza entre seeds (el paper garantiza convergencia desde cualquier punto, no menor dispersión). Las fronteras de decisión de D2 y D3 son topológicamente similares entre sí, verificando el Meta-Teorema 1. La columna derecha de D2/D3 superpone los contornos $P(y = 1|x) = 0{,}5$ solo para runs con acc $\geq 0{,}90$, sobre una nube de puntos de todos los datasets.

6.5. Experimento E: Robustez de ν^* (make_moons)

Objetivo. Verificar que la distribución óptima de parámetros ν^* es robusta a distintas condiciones de entrenamiento: distintas inicializaciones y distintos datasets deben producir distribuciones ν^* similares.

Protocolo. Dos sub-experimentos con $n_{\text{seeds}} = 10, \varepsilon = 0{,}01, 500$ épocas y **SGLD** (`use_sgld=True`, para explorar la distribución de Gibbs ν^* en lugar del MAP):

- **E-1:** `data_seed`$= 42$ fijo, `init_seed` $\in \{0, \ldots, 9\}$ variable.
- **E-2:** `init_seed`$= 4$ fijo, `data_seed` $\in \{0, \ldots, 9\}$ variable.

Magnitud visualizada. Para cada run se extraen los pesos de salida $a_0^m \in \mathbb{R}^2$ de las $M = 64$ neuronas y se calcula la *importancia*:

$$I^m = \|a_0^m\|_2 \,. \tag{44}$$

Las importancias se ordenan de mayor a menor y se representan para cada seed, junto con la media $\pm 1\sigma$ sobre las 10 seeds (figura 1×2):

- **Panel izquierdo (E-1):** importancias con init variable, datos fijos. Banda estrecha $\Rightarrow \nu^*$ es robusta a la inicialización.
- **Panel derecho (E-2):** importancias con datos variables, init fija. Banda estrecha $\Rightarrow \nu^*$ es una propiedad de la geometría del problema, no del dataset concreto.

Si el Meta-Teorema 1 se cumple (minimizador único estable), las curvas de importancia deben ser cualitativamente similares entre seeds: mismo "codo" pronunciado en los primeros ranks y misma caída.

6.6. Experimento F: Simetría de ν^* en `make_circles`

Objetivo. Verificar que en `make_circles` (dataset con simetría rotacional SO(2)) la convergencia del modelo es robusta tanto a variaciones del dataset como de la inicialización.

Protocolo. Dos sub-experimentos con $n_{\text{seeds}} = 10$, $n_{\text{epochs}} = 700$ y $\varepsilon = 0{,}01$:

- **F1:** `init_seed`$= 4$ fija, `data_seed` variable.

- **F2:** `data_seed`= 42 fijo, `init_seed` variable.

Figura. Se muestra un único panel con las curvas de convergencia $J \pm 1\sigma$ para F1 y F2 superpuestas. La banda más estrecha de F1 vs. F2 (o viceversa) indica cuál fuente de variabilidad domina la robustez del entrenamiento.

7. Resumen de la arquitectura

Bloque	Descripción
Datasets	make_moons (A–E) y make_circles (F), $N = 400$, normalizados
ODE	$dX_t/dt = F(X_t, t)$, $t \in [0, 1]$, 10 pasos RK4
Campo F	$(1/M) \sum_m \tanh(a_1^m \cdot x + a_2^m t + b_1^m)\, a_0^m,\ \ M = 64$
Parámetros ODE	$W_1 \in \mathbb{R}^{64\times 3}$, $b_1 \in \mathbb{R}^{64}$, $W_0 \in \mathbb{R}^{2\times 64}$ (384 total)
Clasificador	$\ell = W \cdot X_T + b,\ \ W \in \mathbb{R}^{1\times 2},\ \ b \in \mathbb{R}$ (3 total)
Total parámetros	387
Pérdida	$J = \mathrm{BCE} + \varepsilon \cdot \mathcal{E},\ \ \varepsilon = 0{,}01$
Optimizador (A, D)	Adam [3], $\alpha_0 = 0{,}01$, cosine annealing
Optimizador (C)	SGD, $\alpha = 0{,}01$ constante, sin ruido
Optimizador (B, E, F)	SGLD [2], $\theta \leftarrow \theta - \eta \nabla J + \sqrt{2\eta\varepsilon}\,\xi$
Gradient clip	$G_{\text{máx}} = 5$ (todos los modos)

Referencias

[1] S. Daudin and F. Delarue, *Genericity of the Polyak-Łojasiewicz inequality for mean-field Neural ODEs with entropic regularization*, arXiv:2507.08486, 2025.

[2] M. Welling and Y. W. Teh, *Bayesian Learning via Stochastic Gradient Langevin Dynamics*, in *International Conference on Machine Learning (ICML)*, 2011.

[3] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, in *International Conference on Learning Representations (ICLR)*, 2015. https://arxiv.org/abs/1412.6980

[4] I. Loshchilov and F. Hutter, *SGDR: Stochastic Gradient Descent with Warm Restarts*, in *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1608.03983

[5] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, *Neural Ordinary Differential Equations*, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. https://arxiv.org/abs/1806.07366

[6] B. T. Polyak, *Gradient methods for the minimisation of functionals*, *USSR Computational Mathematics and Mathematical Physics*, 3(4):864–878, 1963.

[7] S. Łojasiewicz, *Une propriété topologique des sous-ensembles analytiques réels*, in *Colloques Internationaux du CNRS*, 117:87–89, 1963.

[8] C. Villani, *Optimal Transport: Old and New*, Springer, 2009.